



Пошаговое руководство по глубокому обучению с помощью PyTorch

Автор: Даниэль Фойгт Годой

Тип файла: pdf

Размер: 33,6 МБ

Язык: английский

Страниц: 1045

Пошаговое руководство по глубокому обучению с помощью PyTorch: полное руководство для инженеров от начального до профессионального уровня 🤖🔥

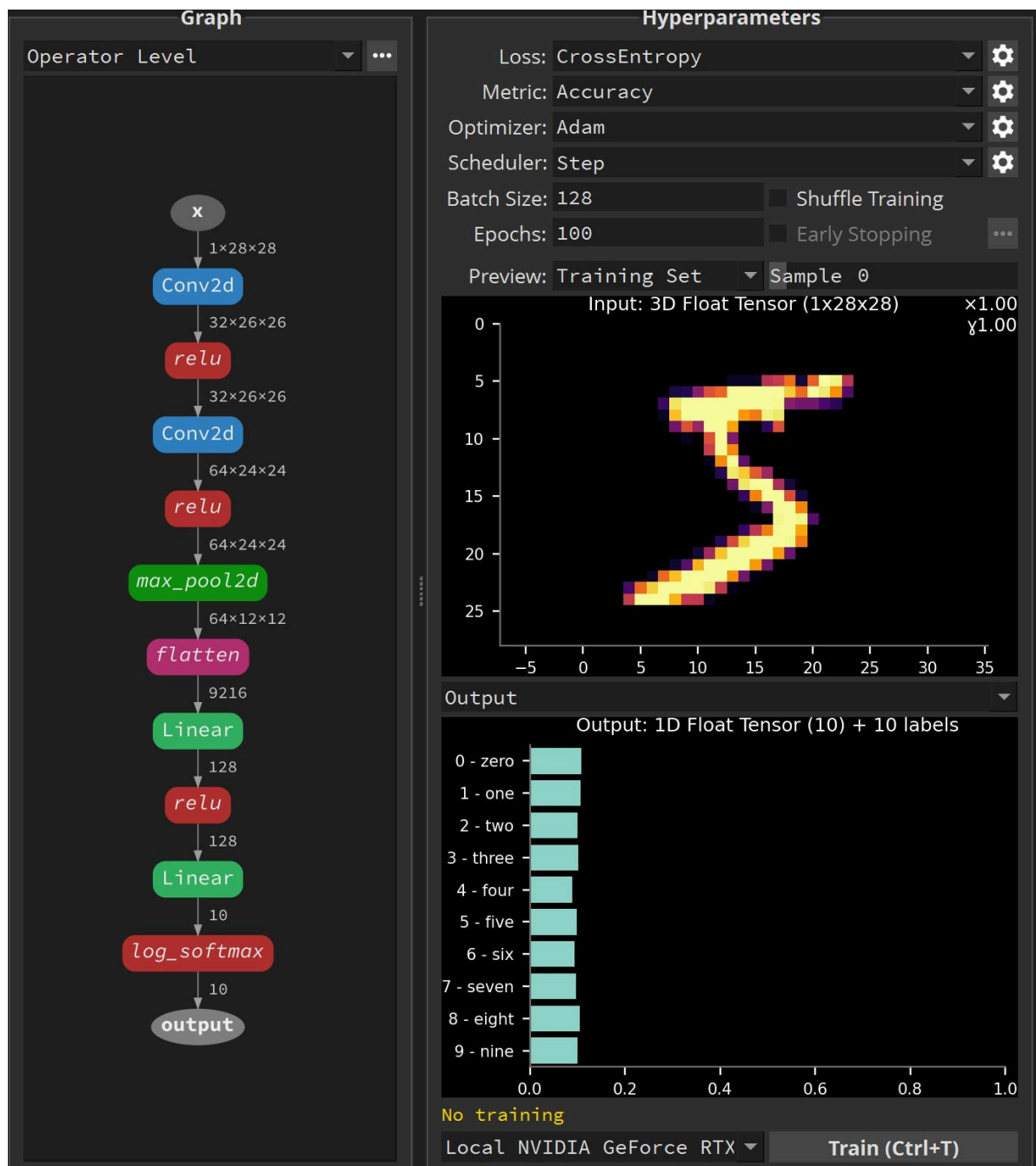
Введение 🚀

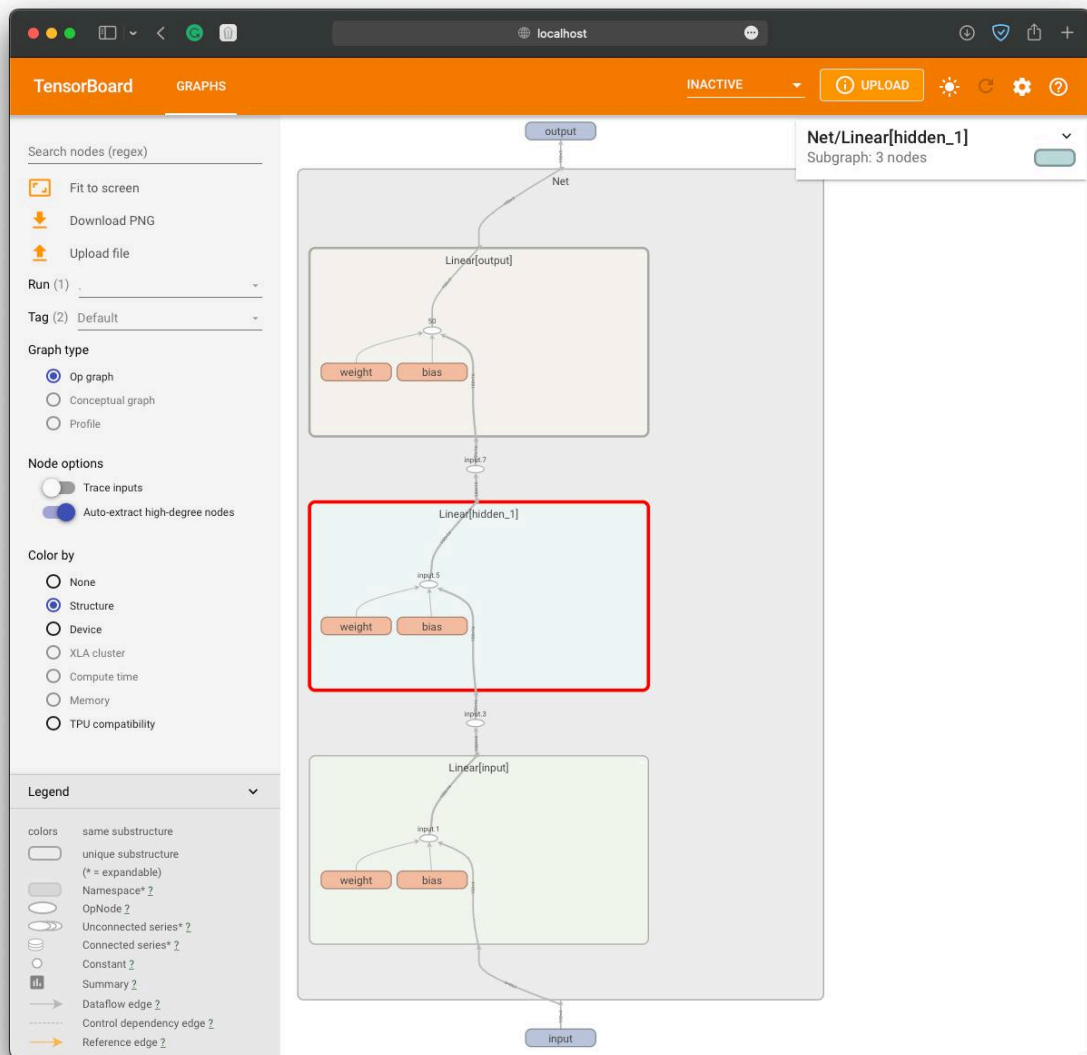
Искусственный интеллект произвел революцию в современной инженерии, здравоохранении, финансах, робототехнике, автономных транспортных средствах, кибербезопасности и научных исследованиях. В основе многих из этих прорывов лежит **глубокое обучение** — разновидность машинного обучения, которая позволяет компьютерам выявлять сложные закономерности в огромных массивах данных.

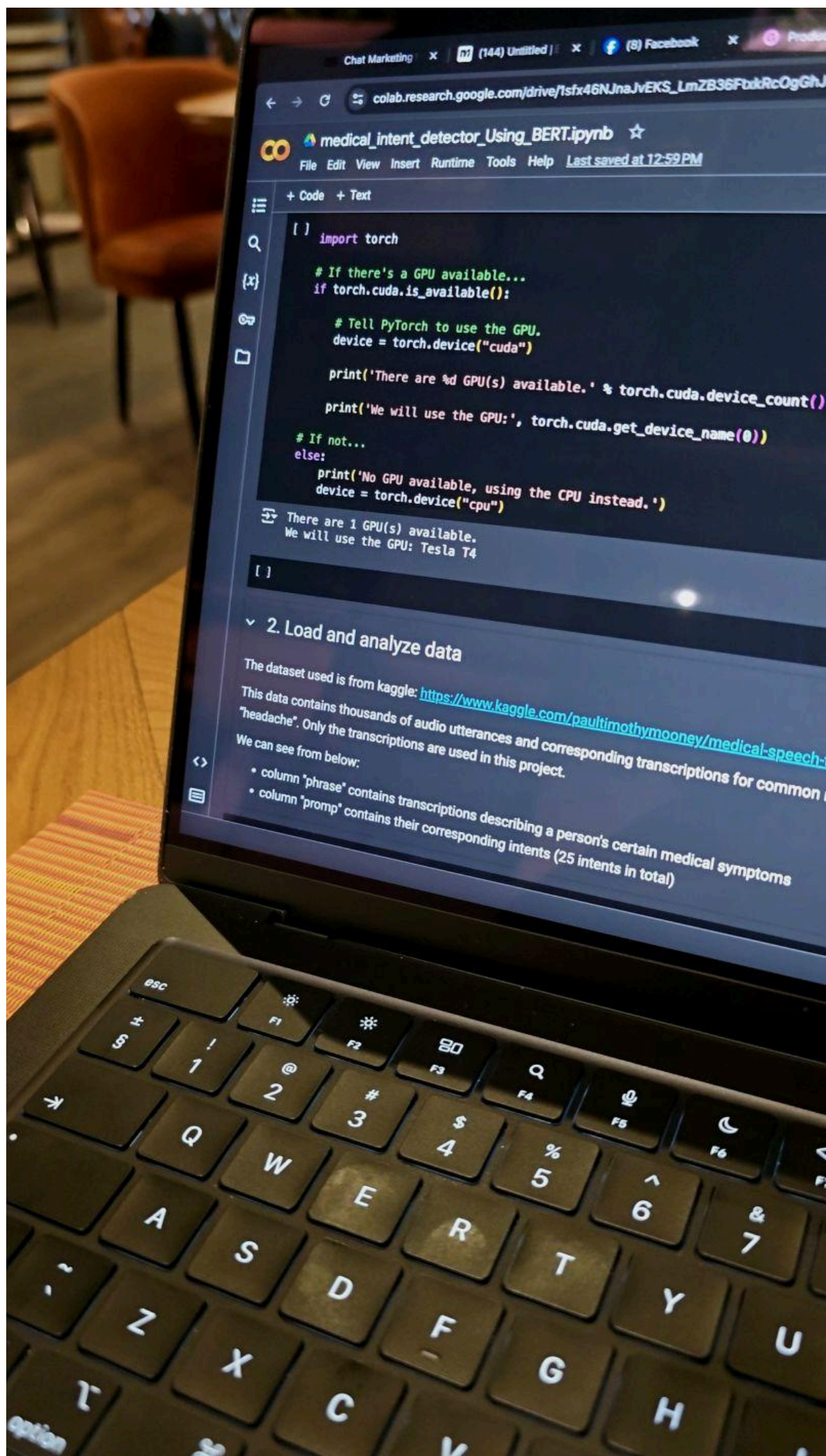
Среди всех фреймворков глубокого обучения **PyTorch** стал одной из самых надёжных платформ, используемых инженерами, исследователями, университетами и ведущими технологическими компаниями по всему миру. PyTorch, разработанный с учётом гибкости, читабельности и скорости, позволяет разработчикам создавать сложные нейронные сети, сохраняя при этом простоту Python.

Независимо от того, являетесь ли вы студентом, впервые столкнувшимся с искусственным интеллектом, или опытным инженером-программистом, создающим системы промышленного уровня, изучение PyTorch станет отличной основой для разработки современных ИИ-систем.

В этом подробном руководстве подробно рассматривается глубокое обучение с использованием PyTorch — от базовых принципов до практической реализации, инженерных концепций, сравнений, диаграмм, примеров, проблем, стратегий оптимизации и промышленных применений.







colab.research.google.com/drive/1sf46NjNaJvEKS_LmZB36F0dkRcOgGhJ

medical_intent_detector_Using_BERT.ipynb ☆

File Edit View Insert Runtime Tools Help Last saved at 12:59 PM

+ Code + Text

```
[ ] import torch

# If there's a GPU available...
if torch.cuda.is_available():

    # Tell PyTorch to use the GPU.
    device = torch.device("cuda")

    print('There are %d GPU(s) available.' % torch.cuda.device_count())
    print('We will use the GPU:', torch.cuda.get_device_name(0))

# If not...
else:
    print('No GPU available, using the CPU instead.')
    device = torch.device("cpu")
```

There are 1 GPU(s) available.
We will use the GPU: Tesla T4

[]

2. Load and analyze data

The dataset used is from kaggle: <https://www.kaggle.com/paultimothymooney/medical-speech-headache>. This data contains thousands of audio utterances and corresponding transcriptions for common "headache". Only the transcriptions are used in this project.

We can see from below:

- column "phrase" contains transcriptions describing a person's certain medical symptoms
- column "prompt" contains their corresponding intents (25 intents in total)



Теоретическая основа

Глубокое обучение основано на биологической структуре человеческого мозга.

Искусственные нейронные сети состоят из взаимосвязанных нейронов, которые обрабатывают информацию слой за слоем.

Типичная модель глубокого обучения содержит:

-  Входной слой
-  Скрытые слои
-  Выходной слой

Каждый нейрон выполняет математические операции:

- Получает входные данные
- Применяет весовые коэффициенты
- Добавляет смещение
- Пропускает результат через функцию активации
- Выдает результат

Во время обучения сеть постоянно обновляет свои весовые коэффициенты, чтобы уменьшить количество ошибок прогнозирования.

Чем больше сеть, тем более сложные закономерности она может выявлять.

Современные возможности глубокого обучения:

- 🤖 Чат-боты
- 🚗 Беспилотные автомобили
- 🩺 Медицинская диагностика
- 📺 Видеоанализ
- 🛰️ Спутниковая визуализация
- 🛍️ Рекомендательные системы
- 🎮 Игровой ИИ
- 📈 Финансовое прогнозирование

Определение

PyTorch — это фреймворк глубокого обучения с открытым исходным кодом, основанный на Python и Torch.

Он предоставляет:

- Динамические вычислительные графы
- Автоматическое дифференцирование (Autograd)
- Ускорение на графическом процессоре
- Операции с тензорами
- Модули нейронных сетей
- Алгоритмы оптимизации
- Утилиты для загрузки данных
- Инструменты для развертывания моделей

PyTorch позволяет инженерам быстро создавать прототипы, обучать, оценивать и внедрять модели нейронных сетей.

Understanding the Core Components

Tensors

Tensors are multidimensional arrays similar to NumPy arrays but capable of running efficiently on GPUs.

Examples include:

- Scalar (0D)
- Vector (1D)
- Matrix (2D)
- Higher-dimensional tensors

Almost every PyTorch operation revolves around tensors.

Autograd

One of PyTorch's most powerful features is **Automatic Differentiation**.

Instead of manually calculating gradients, PyTorch automatically computes derivatives required during backpropagation.

Benefits include:

- Less coding
- Fewer mathematical mistakes
- Faster experimentation

Neural Network Module

The **torch.nn** package provides ready-made components including:

- Linear Layers
- Activation Functions
- Dropout
- Batch Normalization
- Convolution Layers
- Pooling Layers
- Loss Functions

These building blocks simplify model construction.

Optimizers

Optimizers update model parameters.

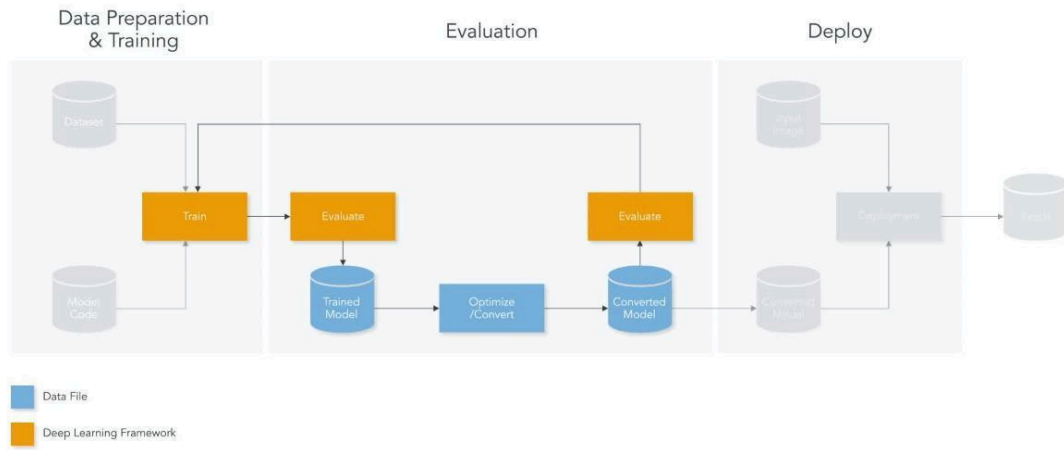
Popular choices include:

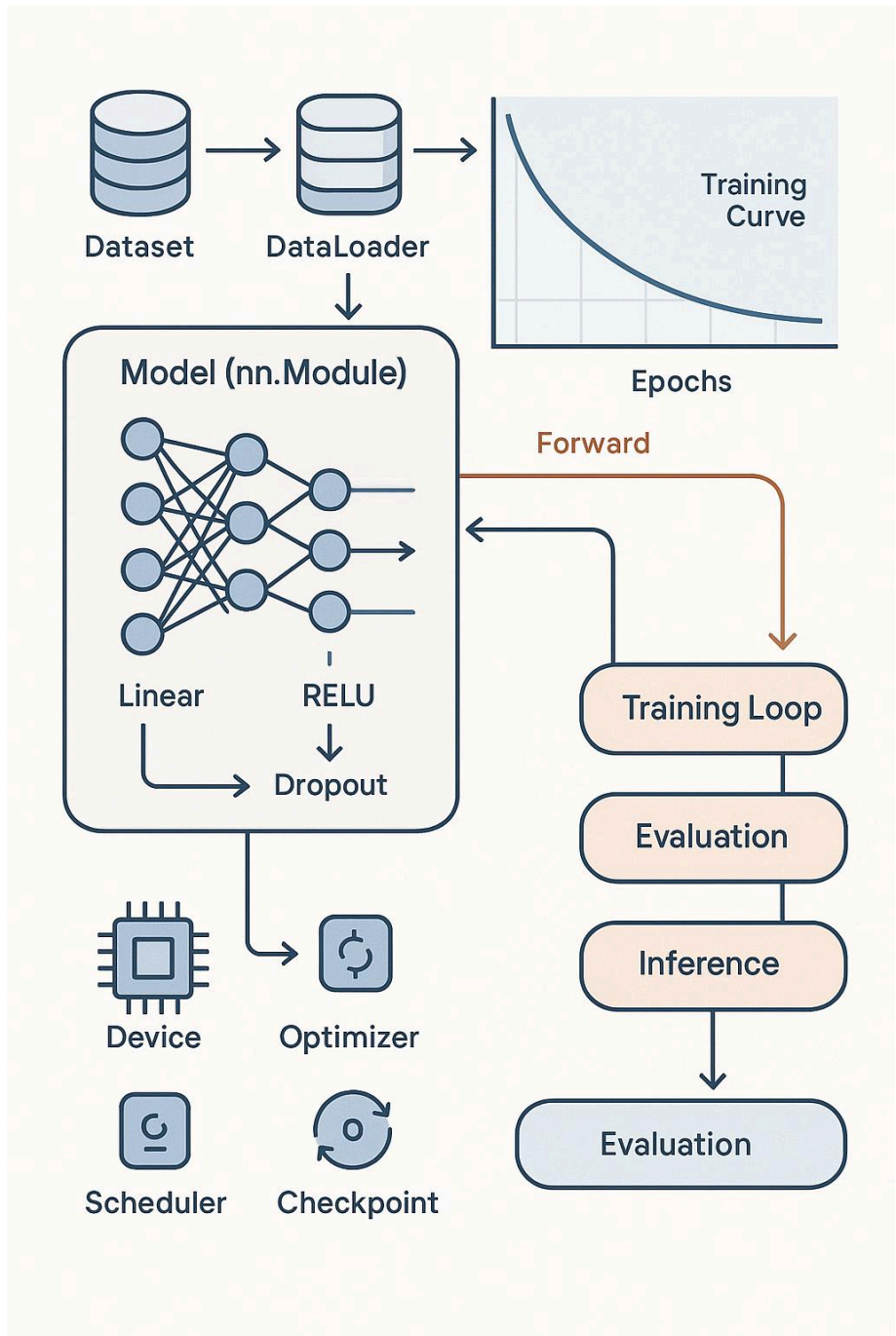
- SGD
- Adam
- RMSprop
- AdamW

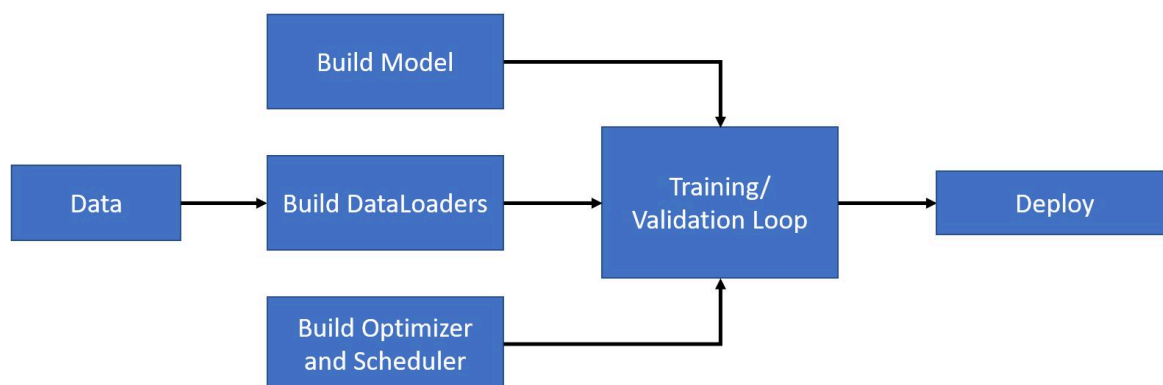
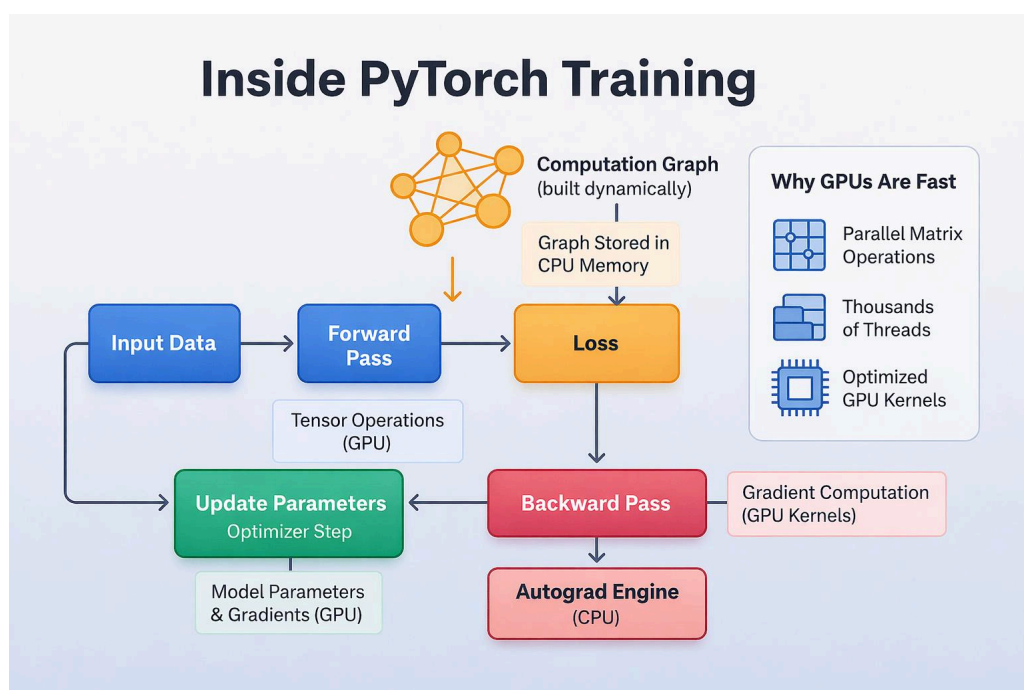
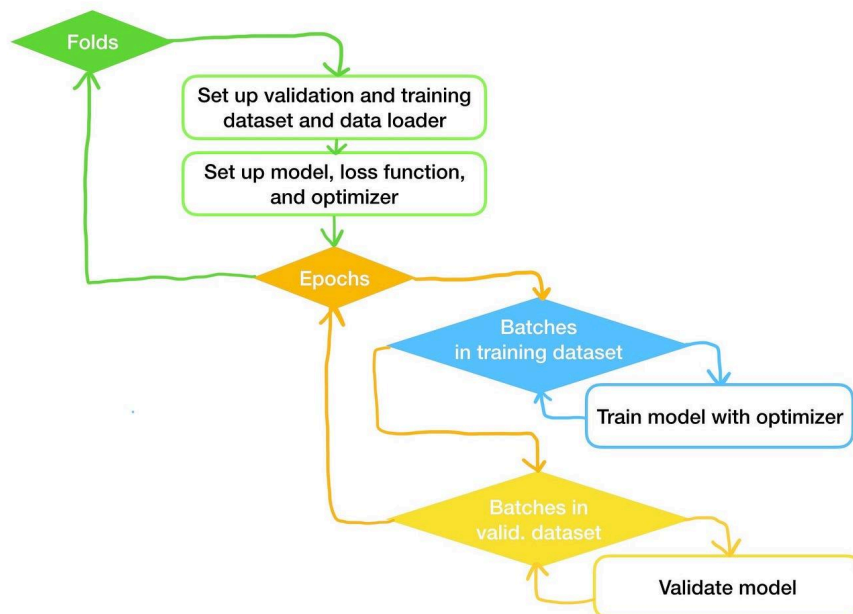
They help the model minimize prediction errors efficiently.

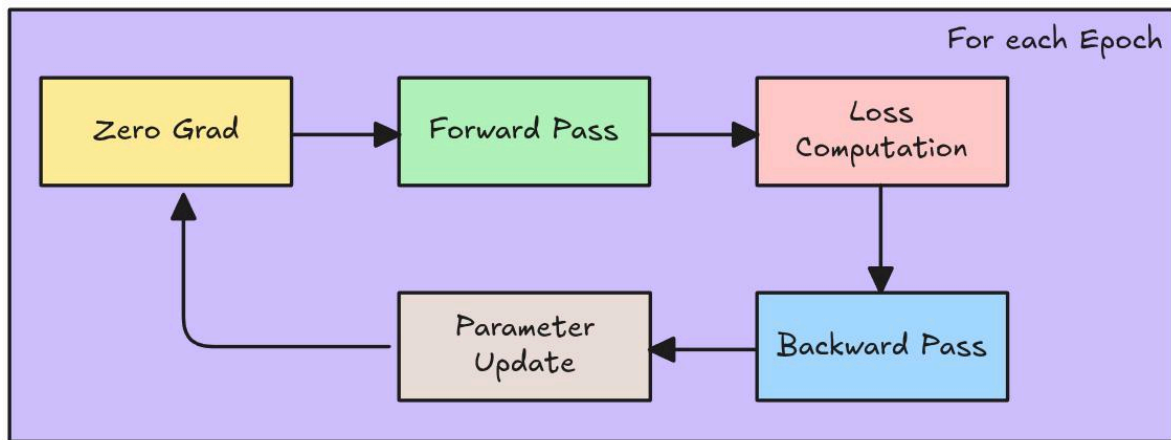
Step-by-Step Deep Learning Workflow

Deep Learning Training, Evaluation & Deployment Flow









Step 1 — Install PyTorch

Install the framework and verify GPU support.

Typical engineering environments include:

- Windows
- Linux
- macOS

Step 2 — Import Libraries

Projects generally begin by importing:

- torch
- torchvision
- numpy
- matplotlib

These libraries provide tensor computation, datasets, and visualization.

Step 3 — Load Dataset

Examples:

- MNIST
- CIFAR-10
- Fashion-MNIST
- ImageNet
- Custom datasets

Proper data loading improves training efficiency.

Step 4 — Data Preprocessing

Typical preprocessing includes:

- Normalization
- Resizing
- Data augmentation
- Random cropping
- Rotation
- Horizontal flipping

Clean data leads to better models.

Step 5 — Build the Neural Network

Define:

- Input size
- Hidden layers
- Activation functions
- Output layer

The architecture depends on the engineering problem.

Step 6 — Select Loss Function

Common choices include:

Task	Loss Function
Classification	CrossEntropyLoss
Regression	MSELoss
Binary Classification	BCELoss
Multi-label	BCEWithLogitsLoss

Step 7 — Choose Optimizer

Popular optimizers:

- Adam
- SGD
- AdamW

Learning rate selection greatly affects convergence.

Step 8 — Train the Model

Training loop:

1. Forward pass
2. Compute loss
3. Backpropagation
4. Update weights
5. Repeat

Training may require hundreds or thousands of epochs.

Step 9 — Evaluate Performance

Important metrics include:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion Matrix

Evaluation determines whether the model generalizes well.

Step 10 — Deploy the Model

Deployment targets include:

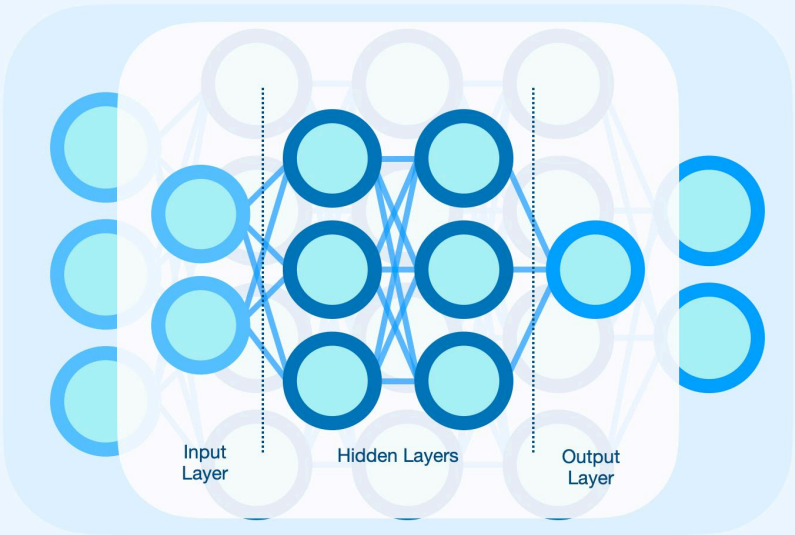
- Cloud servers
- Edge devices
- Mobile apps
- Embedded systems
- Industrial robots

PyTorch vs Other Deep Learning Frameworks

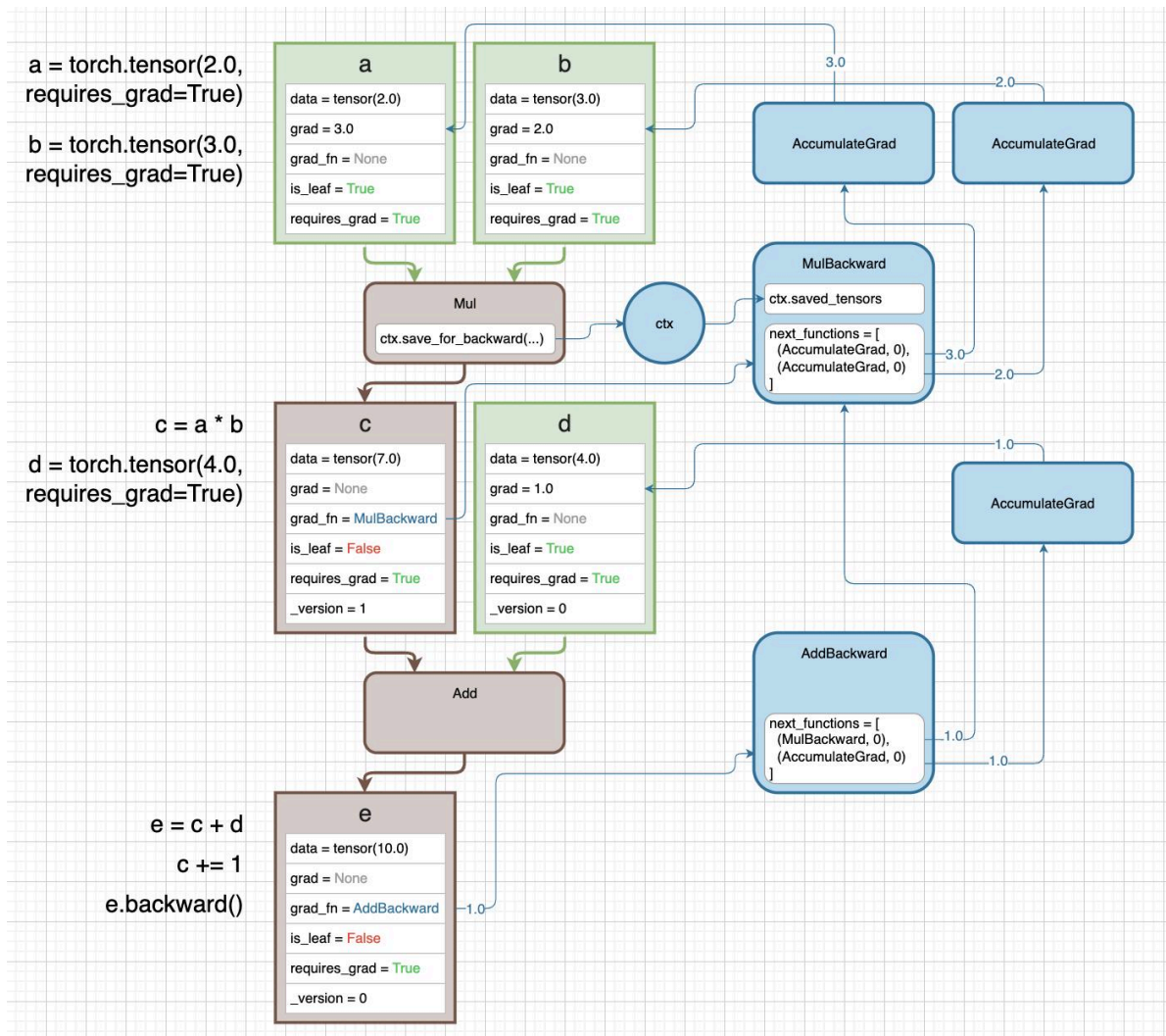
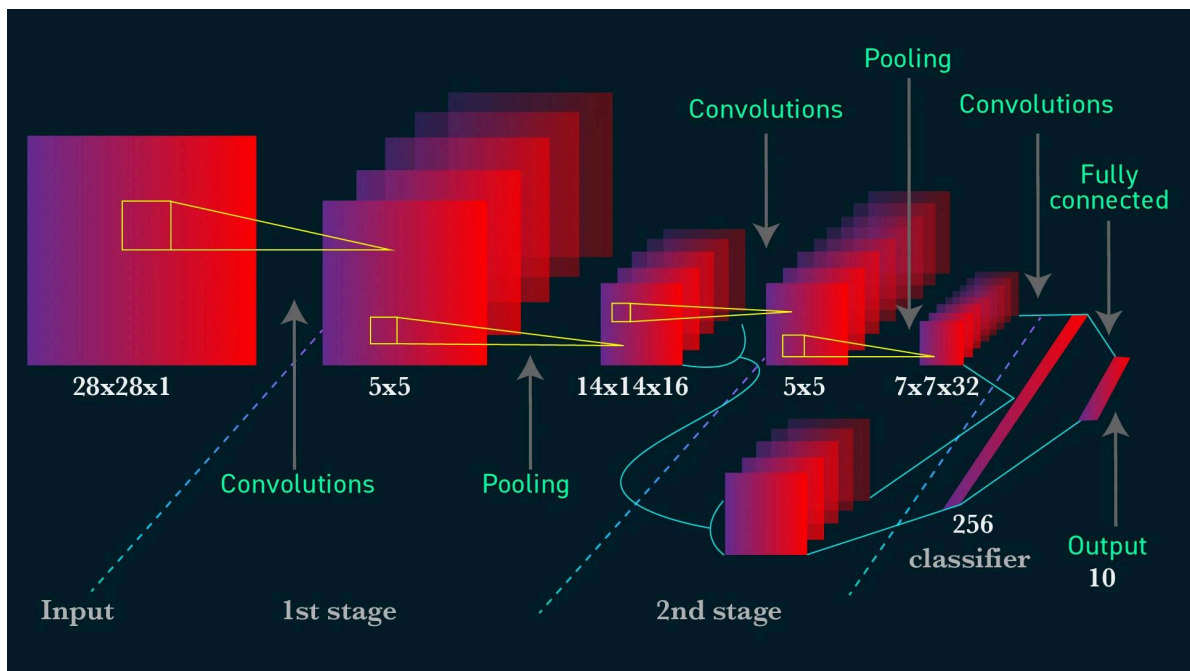
Feature	PyTorch	TensorFlow	Keras
Ease of Learning	★★★★★	★★★	★★★★★
Flexibility	Excellent	High	Medium
Research Popularity	Very High	High	Medium
Deployment	Excellent	Excellent	Good
Dynamic Graph	Yes	Partial	No
GPU Support	Excellent	Excellent	Good
Community	Massive	Massive	Large

Deep Learning Architecture Overview

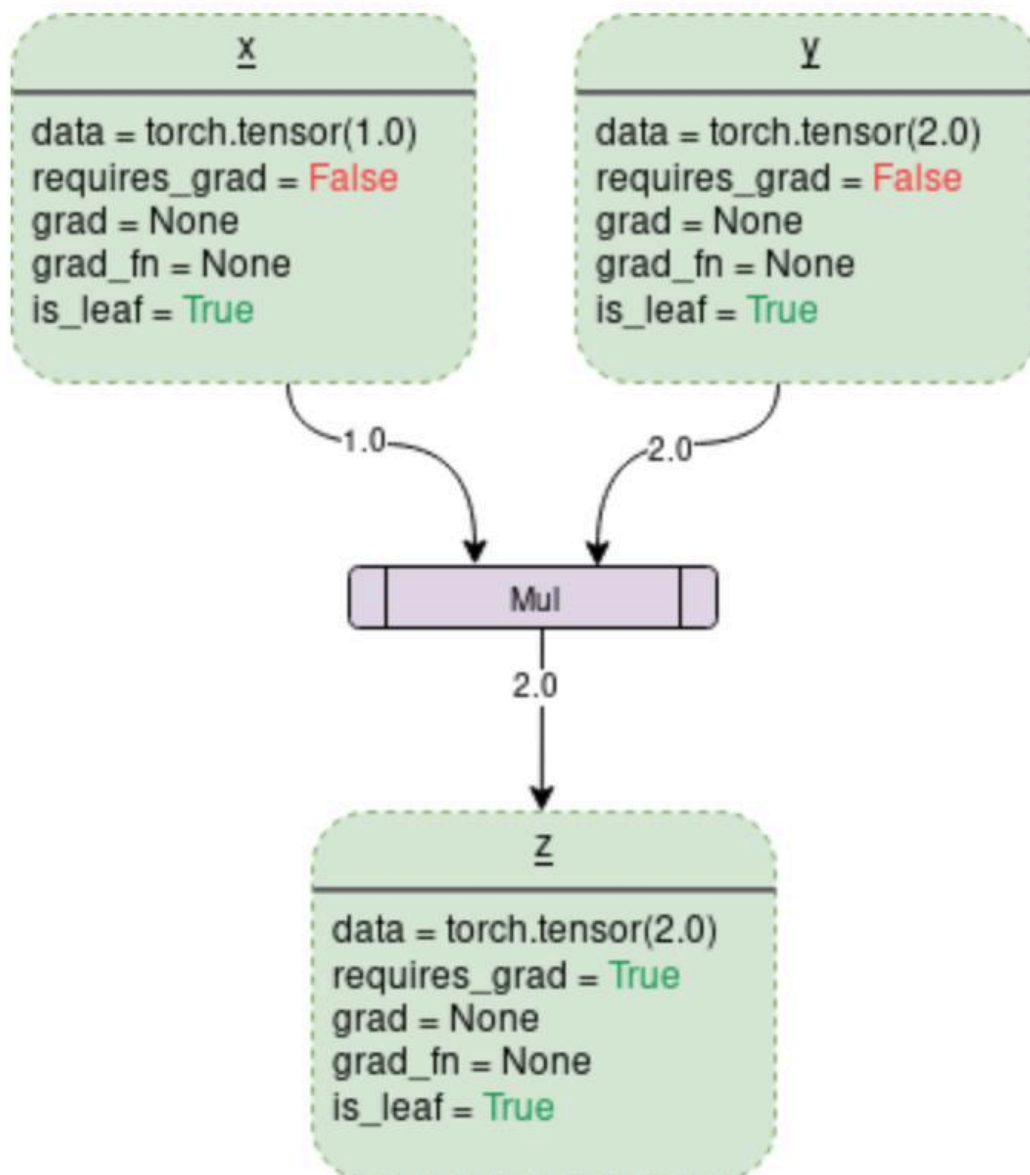
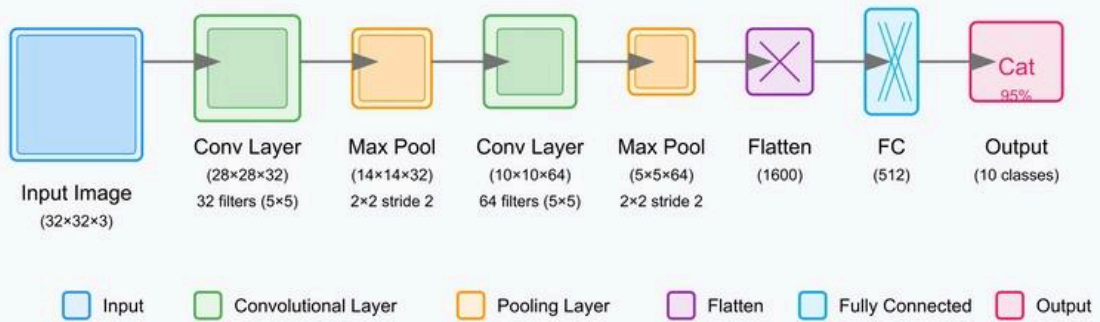
Neural Network Basics



Artificial Neural Network Structure



Convolutional Neural Network (CNN) Architecture



Typical Neural Network Pipeline

Stage	Purpose
Input Layer	Receives data
Hidden Layers	Feature extraction
Activation	Introduces non-linearity
Output Layer	Final prediction

Common Activation Functions

Function	Best Use
ReLU	Most deep networks
Sigmoid	Binary classification
Softmax	Multi-class classification
Tanh	Recurrent networks

Popular Optimizers

Optimizer	Strength
SGD	Stable
Adam	Fast convergence
AdamW	Better regularization
RMSprop	Sequential data

Practical Examples

Image Classification

Input:

- Cat images
- Dog images

Output:

- Cat
- Dog

Used in veterinary systems and smart cameras.

Medical Imaging

Hospitals use CNN models to detect:

- Cancer
- Pneumonia
- Brain tumors
- Retinal diseases

Natural Language Processing

Applications include:

- Translation
- Chatbots
- Document summarization
- Sentiment analysis

Autonomous Vehicles

Deep learning detects:

- Traffic signs
- Pedestrians
- Vehicles
- Road lanes

Industrial Inspection

Factories automatically identify:

- Surface defects

- Cracks
- Missing components

Real-World Engineering Applications

PyTorch powers numerous industries.

Healthcare

- Disease detection
- Medical image segmentation
- Drug discovery

Manufacturing

- Predictive maintenance
- Visual quality inspection
- Industrial automation

Finance

- Fraud detection
- Credit scoring
- Stock prediction

Robotics

- Navigation
- Object recognition
- Motion planning

Agriculture

- Crop disease detection
- Yield estimation
- Smart irrigation

Aerospace

- Satellite image processing
- Aircraft monitoring
- Autonomous drones

Common Mistakes

Beginners frequently encounter these issues:

Ignoring Data Quality

Poor datasets produce poor models.

Using an Incorrect Learning Rate

Too high:

- Training diverges.

Too low:

- Learning becomes extremely slow.

Overfitting

The model memorizes training data instead of learning patterns.

Solutions:

- Dropout
- Regularization
- Data augmentation

Underfitting

Model complexity is insufficient.

Improve by:

- Adding layers
- Increasing neurons
- Training longer

Forgetting Evaluation Mode

Always switch the model into evaluation mode before testing.

Challenges and Solutions

Challenge	Solution
Limited Data	Data augmentation
Slow Training	GPU acceleration
Overfitting	Dropout & Early Stopping
Memory Errors	Smaller batch size
Low Accuracy	Hyperparameter tuning
Class Imbalance	Weighted loss functions

Engineering Case Study

Problem

An automotive manufacturer needed automatic detection of defective engine components.

Solution

Engineers developed a convolutional neural network using PyTorch.

Workflow:

- Dataset collection
- Image preprocessing
- CNN design
- GPU training
- Performance optimization
- Factory deployment

Results

- ✅ 98% inspection accuracy
- ⚡ Inspection speed improved by 6×
- 💰 Production costs reduced by 35%
- 🔍 Human inspection errors significantly decreased

This demonstrates how deep learning delivers measurable engineering value in industrial environments.

Essential Tips ★

Start Simple

Build small neural networks before attempting complex architectures.

Learn Tensor Operations

Tensor manipulation forms the foundation of PyTorch programming.

Understand the Mathematics

Study:

- Linear Algebra
- Calculus
- Probability
- Statistics

Use GPUs

Training speed increases dramatically with CUDA-enabled GPUs.

Visualize Training

Monitor:

- Loss curves
- Accuracy curves
- Validation metrics

Save Checkpoints

Frequently save trained models to prevent losing progress.

Read Documentation

Stay updated with new PyTorch releases and best practices.

Frequently Asked Questions ?

Is PyTorch suitable for beginners?

Yes. Its intuitive Python interface makes it one of the easiest deep learning frameworks to learn.

Do I need advanced mathematics?

Basic linear algebra, calculus, and probability are helpful, but you can begin learning practical PyTorch before mastering every mathematical detail.

Can PyTorch run without a GPU?

Yes. It works on CPUs, although training large models will be considerably slower than on GPU-equipped systems.

Which [programming language](#) is required?

Python is the primary language used with PyTorch and is recommended for nearly all development and research workflows.

Is PyTorch used in industry?

Absolutely. Many technology companies, startups, research laboratories, and engineering organizations use PyTorch for production AI systems and cutting-edge research.

What projects can I build?

You can create image classifiers, object detectors, recommendation systems, chatbots, speech recognition models, anomaly detection systems, robotics applications, and predictive maintenance solutions.

How long does it take to learn PyTorch?

With consistent practice, beginners can understand the fundamentals within a few weeks. Building production-quality models and mastering advanced architectures typically takes several months of hands-on experience.

Conclusion

Deep Learning with PyTorch has become one of the most valuable skills in modern engineering, enabling professionals to solve complex problems across computer vision, natural language processing, robotics, healthcare, finance, manufacturing, and autonomous systems. Its dynamic computation graph, intuitive Python interface, powerful automatic differentiation engine, and extensive ecosystem make it an ideal framework for both education and industrial deployment.

By following a structured learning path—understanding tensors, designing neural networks, preprocessing data, selecting appropriate loss functions and optimizers, training models, evaluating performance, and deploying solutions—you can confidently develop AI applications that deliver real-world impact. Whether your goal is academic research, professional software engineering, or intelligent automation, mastering PyTorch provides a strong foundation for building scalable, efficient, and innovative deep learning systems that meet the growing demands of industries throughout the USA, UK, Canada, Australia, Europe, and beyond.

Related Posts:





Preview PDF Book

[← Previous Book](#)

[Next Book →](#)

EngiVerse

Explore a vast library of free engineering ebooks curated to help you expand your technical knowledge, conduct groundbreaking research, and advance your career. Our mission is to create a hub of knowledge that fosters innovation, empowers aspiring engineers, and bridges the gap between education and industry.

[Home](#) [Books](#) [About Us](#) [Contact](#)
[Privacy Policy](#) [DMCA](#)

copyright 2025 EngiVerse. All Right Reserved